

Aim

To develop a low-cost IoT system that detects an approaching ambulance and **automatically overrides the traffic signal** (turns green for ambulance direction), transmits ambulance location/medical info to the hospital, and logs events to a cloud database — thereby reducing emergency response time

description (how it works)

- The **ambulance unit** reads a siren sensor and GPS, and when siren is active it sends an override message (and ambulance metadata) using a long-range radio (LoRa) or serial radio to the nearby traffic signal controller and the hospital display unit.
- The **traffic unit** receives messages from ambulance(s), overrides the normal traffic cycle to show green in the ambulance direction, displays ambulance metadata (on OLED / LEDs) and then returns to the normal cycle after the ambulance passes.
- The **hospital unit** receives the ambulance message (location & patient info) and displays it for staff readiness; it can also generate a buzzer alert.
- All override events and telemetry are logged to a cloud DB for records/analysis.

Components (concise list)

(From your file — core parts.)

iiot internal

- ESP32 (or equivalent MCU) — ambulance, traffic, hospital nodes
- GPS module (e.g., NEO-6M) for ambulance location
- LoRa TX/RX (or alternative long-range comms)
- Siren sensor (digital input)
- Relay module / LED traffic unit (to drive red/yellow/green)
- OLED display (128×64) or similar for traffic/hospital displays
- Buzzer for audible alert
- Cloud DB (Firebase / MQTT broker + DB) for logs
- Power supplies, wiring, enclosures

What to record (experiment / production records)

Create consistent logs for later analysis — both local (serial log) and cloud DB. Recommended fields and formats:

Event log (CSV / DB document)

- event_id (uuid)
- timestamp_utc (ISO 8601)
- node_type (ambulance / traffic / hospital)
- node_id (e.g., AMB_01 / TRF_01)
- event_type (siren_on, override_sent, override_received, override_activated, override_cleared, gps_fix, heartbeat)
- lat (float)

- lon (float)
- patient_info (structured JSON or short text) — e.g. {blood:"O+",oxy:"Low"}
- signal_state_before (red/yellow/green)
- signal_state_after (green)
- round_trip_latency_ms (if measured)
- notes (free text)

Telemetry / heartbeat (every n seconds)

- node_id, timestamp, available_satellites, battery_voltage, rssi, last_event

Test measurements to record

- detection_time_ms: from siren sensor activation to message TX.
- network_latency_ms: from ambulance TX to traffic RX (average & max).
- override_time_ms: from traffic RX to green ON.
- total_delay_reduction_percent: measured by real route runs comparing baseline vs system ON (see below).
- success_rate: number of successful overrides / total attempts.

Database schema suggestion

- ambulances collection: {id, vehicle_no, crew_contact, medical_profile_template}
- events collection: each event doc = event log fields above
- routes collection (optional): recorded GPS track for run analysis

(These record fields let you compute reliability, latency, and response improvements.)

Testing / validation protocol (short)

1. Baseline: run ambulance along test route with normal signals; measure arrival time to hospital (3 runs averaged).
2. With system: enable system and run same route; measure arrival time (3 runs).
3. Compute $\text{time_saved} = \text{baseline_time} - \text{system_time}$; $\text{time_saved\%} = (\text{time_saved} / \text{baseline_time}) \times 100$.
4. For each test: capture timestamps for siren_on, override_sent, override_received, override_activated, override_cleared. Log heartbeats and packet RSSI.
5. Run stress tests (concurrent ambulances, simulated packet loss) and measure success_rate.

Your PDF reported **30–40% faster arrival times** in prototype tests — keep that as a target figure but reproduce with local tests.

iiot internal

Code (cleaned and grouped)

Below are cleaned versions of the three short code files from your uploaded project (kept functionally equivalent and commented). Use these as `ambulance_unit.ino`, `traffic_unit.ino`, and `hospital_unit.ino`. They are ready for ESP32 Arduino environment.

Note: these are simplified sketches based on your provided code in the uploaded file. For production integrate LoRa libraries, GPS parsing and cloud API calls (not shown here).

Code

`ambulance_unit.ino`

`// ambulance_unit.ino`

`// Reads siren sensor, sends data to traffic and hospital units over Serial (or LoRa)`

`#include <HardwareSerial.h>`

`#define SIREN_PIN 12`

`#define TRAFFIC_PIN 14`

`#define HOSP_PIN 27`

`#define LED_PIN 2`

`#define BUZ_PIN 4`

`HardwareSerial TrafficSerial(2); // TX/RX pins chosen in setup`

`HardwareSerial HospSerial(0);`

`bool sirenState = false;`

`unsigned long lastSirenToggle = 0;`

`void setup() {`

`Serial.begin(115200);`

`TrafficSerial.begin(9600, SERIAL_8N1, 25, 26); // adjust pins`

`HospSerial.begin(9600, SERIAL_8N1, 32, 33);`

`pinMode(SIREN_PIN, INPUT_PULLUP);`

`pinMode(TRAFFIC_PIN, INPUT_PULLUP);`

`pinMode(HOSP_PIN, INPUT_PULLUP);`

`pinMode(LED_PIN, OUTPUT);`

Laboratory record of:
Industrial internet of things and systems lab

Roll no: 160123749012

experiment no:

Sheet No.

Date:

```
ledcAttachPin(BUZ_PIN, 0);

}

String buildData() {

    // Example data payload; replace with real GPS + patient JSON

    return "Blood:O+ Oxy:Low LAT:12.97 LON:77.59\n";

}

void loop() {

    unsigned long now = millis();

    // debounced siren toggle (button style)

    if (!digitalRead(SIREN_PIN) && now - lastSirenToggle > 300) {

        lastSirenToggle = now;

        sirenState = !sirenState;

    }

    // manual send to traffic

    if (!digitalRead(TRAFFIC_PIN) && now - lastSirenToggle > 300) {

        TrafficSerial.print(buildData());

    } // manual send to hospital

    if (!digitalRead(HOSP_PIN) && now - lastSirenToggle > 300) {

        HospSerial.print(buildData());

    }

    // audible tone while siren state active

    static int f = 700;

    static bool up = true;

    if (sirenState) {

        f += up ? 50 : -50;
```

Laboratory record of:
Industrial internet of things and systems lab

Roll no: 160123749012

experiment no:

Sheet No.

Date:

```
if (f > 1200 || f < 700) up = !up;

ledcWriteTone(0, f);

digitalWrite(LED_PIN, HIGH);

} else {

    noTone(BUZ_PIN);

    digitalWrite(LED_PIN, LOW);

}delay(20);

}
```

traffic_unit.ino

// traffic_unit.ino
// Receives ambulance messages and displays them on OLED; drives LED indicators

```
#include <HardwareSerial.h>
#include <Adafruit_SSD1306.h>
```

```
#define BLOOD_LED 16
#define OXY_LED 17
```

```
HardwareSerial AmbulanceSerial(2);
Adafruit_SSD1306 display(128, 64, &Wire, -1);
```

```
void setup() {
    Serial.begin(115200);
    AmbulanceSerial.begin(9600, SERIAL_8N1, 33, 32);
    pinMode(BLOOD_LED, OUTPUT);
    pinMode(OXY_LED, OUTPUT);
    display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
    display.clearDisplay();
    display.display();
}
```

```
void loop() {
    if (AmbulanceSerial.available()) {
        String msg = AmbulanceSerial.readStringUntil('\n');
        // Light indication to show arrival data
        digitalWrite(BLOOD_LED, HIGH);
        digitalWrite(OXY_LED, HIGH);

        // Show on OLED
        display.clearDisplay();
        display.setCursor(0,0);
        display.println(msg);
        display.display();

        delay(3000); // visible time
        digitalWrite(BLOOD_LED, LOW);
        digitalWrite(OXY_LED, LOW);
    }
}
```

Laboratory record of:
Industrial internet of things and systems lab

Roll no: 160123749012

experiment no:

Sheet No.

Date:

hospital_unit.ino

// hospital_unit.ino
// Receives ambulance data, displays and sounds buzzer, runs normal traffic LEDs when idle

```
#include <HardwareSerial.h>
```

```
#define R_LED 16  
#define Y_LED 17  
#define G_LED 18  
#define BUZZER 4
```

```
HardwareSerial AmbulanceSerial(1);
```

```
bool sirenActive = false;  
unsigned long sirenStart = 0;  
int state = 0;  
unsigned long lastStateChange = 0;
```

```
void setup() {  
  Serial.begin(115200);  
  AmbulanceSerial.begin(9600, SERIAL_8N1, 26, 25);  
  pinMode(R_LED, OUTPUT);  
  pinMode(Y_LED, OUTPUT);  
  pinMode(G_LED, OUTPUT);  
  pinMode(BUZZER, OUTPUT);  
}
```

```
void setLights(bool r, bool y, bool g) {  
  digitalWrite(R_LED, r);  
  digitalWrite(Y_LED, y);  
  digitalWrite(G_LED, g);  
}
```

```
void loop() {  
  unsigned long now = millis();
```

```
  if (AmbulanceSerial.available()) {  
    String m = AmbulanceSerial.readStringUntil('\n');  
    sirenActive = true;  
    sirenStart = now;  
    setLights(false, false, true); // set green while ambulance incoming  
    tone(BUZZER, 800);  
    // Optionally display 'm' on display (not shown)  
  }
```

```
  if (sirenActive && now - sirenStart > 5000) {  
    sirenActive = false;  
    noTone(BUZZER);  
  }
```

```
  // Normal traffic cycling when not in siren mode  
  if (!sirenActive) {  
    if (state == 0) {  
      setLights(true, false, false);  
      if (now - lastStateChange > 3000) { state = 1; lastStateChange = now; }  
    } else if (state == 1) {  
      setLights(false, true, false);
```



```
if (now - lastStateChange > 1000) { state = 2; lastStateChange = now; }  
} else {  
  setLights(false, false, true);  
  if (now - lastStateChange > 3000) { state = 0; lastStateChange = now; }  
}  
}  
}
```

Expected outputs

Use the **Event log** schema above and also log serial lines for debugging. Example serial prints:

Ambulance serial (TX):

[2025-11-10T15:23:12Z] EVENT: siren_on id=AMB_01 gps=12.9701,77.5946 patient={"blood":"O+","oxy":"Low"}
[2025-11-10T15:23:12Z] SENT -> TRF_01 rssi=-65

Traffic unit (RX)

[2025-11-10T15:23:12Z] RX from AMB_01 payload="Blood:O+ Oxy:Low LAT:12.97 LON:77.59"
[2025-11-10T15:23:12Z] OVERRIDE_ACTIVATED -> setGreen(dir=AMB_01) latency_ms=120
[2025-11-10T15:23:20Z] OVERRIDE_CLEARED -> returned to normal cycle

Out puts

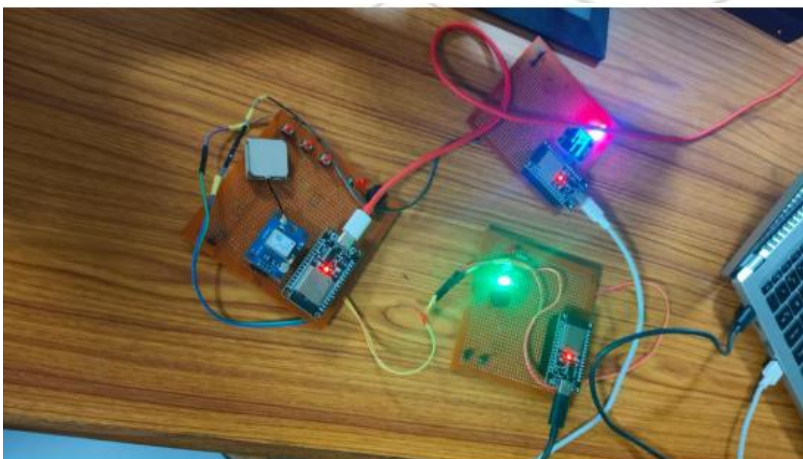


The screenshot shows the Arduino IDE interface. The top toolbar includes icons for checking, running, and uploading code, along with a dropdown menu set to 'Arduino BT'. The left sidebar shows a file explorer with 'ambulance.ino' selected. The main editor displays the following code:

```
19 // --- Siren / LED ---  
20 bool toneState = false;  
21 int toneFreq = 700;  
22 bool toneUp = true;  
23 unsigned long lastToneUpdate = 0;  
24 unsigned long lastBlink = 0;  
25 unsigned long blinkInterval = 300;  
26  
27 // --- PWM setup ---  
28 const int channel = 0;  
29 const int resolution = 8;
```

The 'Output' window at the bottom shows the following message:

```
Sketch uses 1868 bytes (6%) of program storage space. Maximum is 28672 bytes.  
Global variables use 376 bytes (18%) of dynamic memory, leaving 1672 bytes for local variables. Maximum is 2048 bytes.
```



Laboratory record of:
Industrial internet of things and systems lab

Roll no: 160123749012
experiment no:
Sheet No.
Date:

